

ScalaCheck is just one property-based testing library. And while there's nothing wrong with it, we'll derive our own library in this chapter, starting from scratch. Like in chapter 7, this is mostly for pedagogical purposes, but also partly because we should consider no library to be the final word on any subject. There's certainly nothing wrong with using an existing library like ScalaCheck, and existing libraries can be a good source of ideas. But even if you decide you like the existing library's solution, spending an hour or two playing with designs and writing down some type signatures is a great way to learn more about the domain and understand the design trade-offs.

8.2 Choosing data types and functions

This section will be another messy and iterative process of discovering data types and functions for our library. This time around, we're designing a library for property-based testing. As before, this is a chance to peer over the shoulder of someone working through possible designs. The particular path we take and the library we arrive at isn't necessarily the same as what you would come up with on your own. If property-based testing is unfamiliar to you, even better; this is a chance to explore a new domain and its design space, and make your own discoveries about it. If at any point you're feeling inspired or have ideas of your own about how to design a library like this, don't wait for an exercise to prompt you—*put the book down* and go off to play with your ideas. You can always come back to the chapter if you run out of ideas or get stuck.

8.2.1 Initial snippets of an API

With that said, let's get started. What data types should we use for our testing library? What primitives should we define, and what might they mean? What laws should our functions satisfy? As before, we can look at a simple example and “read off” the needed data types and functions, and see what we find. For inspiration, let's look at the ScalaCheck example we showed earlier:

```
val intList = Gen.listOf(Gen.choose(0,100))
val prop =
  forAll(intList)(ns => ns.reverse.reverse == ns) &&
  forAll(intList)(ns => ns.headOption == ns.reverse.lastOption)
```

Without knowing anything about the implementation of `Gen.choose` or `Gen.listOf`, we can guess that whatever data type they return (let's call it `Gen`, short for *generator*) must be parametric in some type. That is, `Gen.choose(0,100)` probably returns a `Gen[Int]`, and `Gen.listOf` is then a function with the signature `Gen[Int] => Gen[List[Int]]`. But since it doesn't seem like `Gen.listOf` should care about the type of the `Gen` it receives as input (it would be odd to require separate combinators for creating lists of `Int`, `Double`, `String`, and so on), let's go ahead and make it polymorphic:

```
def listOf[A](a: Gen[A]): Gen[List[A]]
```

We can learn many things by looking at this signature. Notice what we're *not* specifying—the size of the list to generate. For this to be implementable, our generator must therefore either assume or be told the size. Assuming a size seems a bit inflexible—any